Autenticación segura por medio del modelo User de Django

Luis Diego Soto — Alejandro Valverde



Análisis del Sistema Actual

Estado del Sistema Actual

Acceso a base de datos

Determina si el usuario existe en la base de datos



Otorga permisos persistentes

Si el usuario existe, su ID le da acceso persistente al sistema.



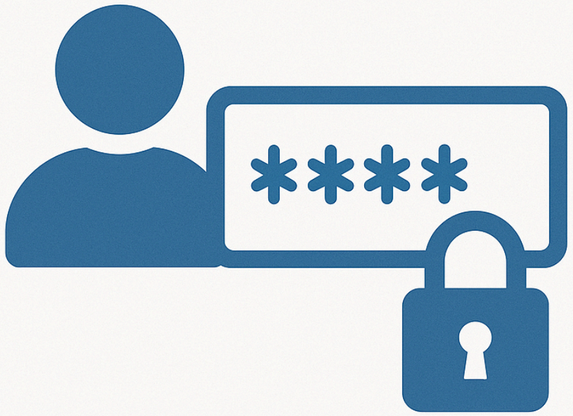


Riesgo Principal

Persistencia de sesión

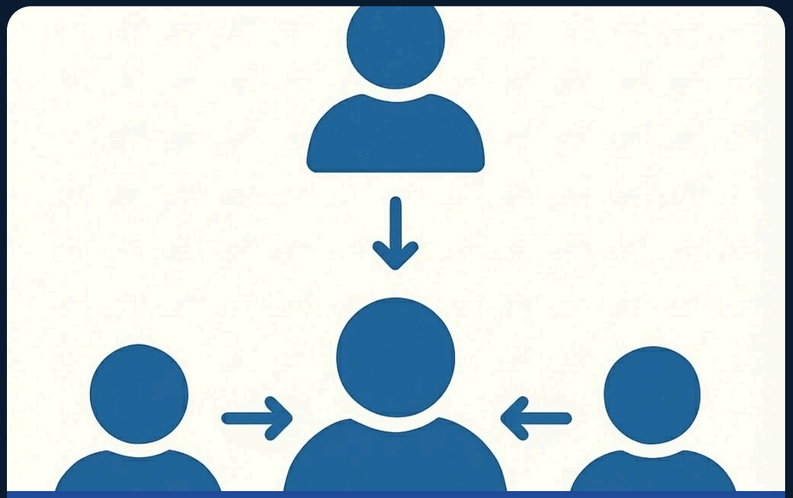
Imposibilita la desconexión de sesiones existentes.

La Solución



Utilizar el sistema de autenticación nativo de Django.

Implica



Unificar el sistema de usuarios de EIEInfo bajo el modelo User de Django.

Decisiones de diseño

```python
class Funcionario(PolymorphicModel):
    # Usuario
    usuario = models.OneToOneField(User, null=True, blank=True,
                                   on_delete=models.SET_NULL)
```

Se utiliza el modelo base de User

Se confeccionarion scripts encargados de popular el nuevo modelo en base al anterior.

```python
class IsEstudiante(Specification):
    """Verifica que el usuario sea un Estudiante."""

    def __init__(self, requiere_activo: bool = True):
        """
        Args:
            requiere_activo: Si True, verifica que la cuenta esté activa
        """
        self.requiere_activo = requiere_activo

    def is_satisfied_by(self, user, **kwargs) -> Tuple[bool, Dict[str, Any]]:
        if not user.is_authenticated:
            return False, {}

        try:
            estudiante = user.estudiante

            if self.requiere_activo and not estudiante.activo:
                return False, {'estudiante': estudiante, 'cuenta_inactiva': True}

            return True, {'estudiante': estudiante}
        except Estudiante.DoesNotExist:
            pass

        return False, {}
```

Specification Pattern para crear decoradores escalables

- Encapsula reglas de negocio en objetos pequeños, reutilizables y recombinables.
- Se pueden combinar estas reglas para generar permisos más complejos

```python
def revisor_or_comision_tfg_login_required(view_func):
    """
    Permite acceso a:
      - Encargados de comisiones TFG
      - Miembros de comisiones TFG
      - Revisores asignados al TFG (si viene tfg_id)
    Inyecta:
      - request.es_encargado_tfg
      - request.es_miembro_tfg
      - request.es_revisor_tfg
    """
    def _wrapped_view(request, *args, **kwargs):
        prof_id = request.session.get('prof_id')

        if not prof_id:
            messages.info(request, "Debe iniciar sesión.")
            return redirect(f"{reverse_lazy('prof_login')}?next={request.path}")

        try:
            profesor = Profesor.objects.get(id=prof_id)
        except Profesor.DoesNotExist:
            messages.error(request, "Perfil de profesor no encontrado.")
            return redirect('prof_home')


        # Buscar TODAS las comisiones TFG tolerando singular/plural y acento/no acento
        nombre_tfg_q = Q(nombre__icontains="Trabajos Finales de Graduación")
        comisiones_tfg = Comision.objects.filter(nombre_tfg_q)

        # ¿Es encargado? (encargado es FK a Profesor → comparar con profesor.id)
        es_encargado = comisiones_tfg.filter(encargado=profesor).exists()

        # ¿Es miembro? (miembros es M2M a Funcionario → comparar con funcionario_ptr)
        es_miembro = comisiones_tfg.filter(miembros=profesor.funcionario_ptr).exists()


        # ¿Es revisor del TFG específico?
        tfg_id = kwargs.get('tfg_id')
        es_revisor = False
        if tfg_id:
            es_revisor = RevisionRevisor.objects.filter(tfg_id=tfg_id, revisor=profesor).exists()
        else:
            pass

        # Inyectar flags en request
        request.es_encargado_tfg = es_encargado
        request.es_miembro_tfg  = es_miembro
        request.es_revisor_tfg  = es_revisor

        if not (es_encargado or es_miembro or es_revisor):
            messages.warning(request, "No tiene permisos para esta sección.")
            return redirect('prof_home')

        return view_func(request, *args, **kwargs)

    return _wrapped_view
```

```python
revisor_or_comision_tfg_login_required = (
    IsProfesor() & (InComision('Trabajos Finales de Graduación') | IsRevisorTFG())
).as_decorator(
    redirect_url='prof_home',
    denied_message="No tiene permisos para esta sección."
)
```

Módulo de servicios de usuario

```python
class BaseRoleLoginForm(forms.Form):
    """Formulario base para login con validación de roles."""

    email = forms.EmailField(label="Correo Electrónico")

    # El nombre del campo ahora debe de ser "password"
    password = forms.CharField(widget=forms.PasswordInput, label="Contraseña")

    required_role = None  # Debe ser definido en subclases

    def __init__(self, request=None, *args, **kwargs):
        self.request = request
        super().__init__(*args, **kwargs)

    def clean(self):
        cleaned = super().clean()
        email = cleaned.get("email")
        password = cleaned.get("password")

        if not email or not password:
            return cleaned

        # Autenticar con Django
        user = authenticate(
            self.request,
            username=email,
            password=password,
        )

        if user is None:
            raise ValidationError("Correo o contraseña incorrectos")

        # Validar rol requerido
        if self.required_role and not hasattr(user, self.required_role):
            raise ValidationError("Usted no tiene permisos para este tipo de acceso.")

        # Validaciones específicas del perfil
        self.validate_profile(user)

        cleaned["user"] = user
        return cleaned

    def validate_profile(self, user):
        """
        Espacio para validaciones adicionales.
        Override en subclases si es necesario.
        En legacy estudiantes y profesor deben checkear estado activo. Puede que se cambie a futuro.
        """
        pass

    def get_user(self):
        """Retorna el usuario autenticado."""
        return self.cleaned_data.get("user")
```

```python
class LoginProfesorForm(BaseRoleLoginForm):
    """Formulario para verificar las credenciales de profesor con Django Auth."""
    required_role = 'funcionario'

    def validate_profile(self, user):
        """Validar que sea específicamente un Profesor activo."""
        from profesores.models import Profesor

        funcionario = user.funcionario

        if not isinstance(funcionario, Profesor):
            raise forms.ValidationError(
                "Usted no tiene permisos para este tipo de acceso."
            )

        if not funcionario.estaActivo:
            raise forms.ValidationError(
                "Usted no tiene nombramientos asignados en el ciclo"
                " actual, por favor contacte a su director de "
                "departamento."
            )

    def save(self, request):
        """Realiza el login y configura la sesión."""
        user = self.get_user()
        profesor = user.funcionario

        django_login(request, user)

        # Sesión igual al sistema original
        request.session['prof_id'] = profesor.id
        request.session['last_login'] = str(datetime.now())
        request.session['is_logged'] = True
        request.session.set_expiry(
            settings.CONFIG['sesiones']['profesores']
        )
```

Mejoras a futuro

Unificación del Sistema de Usuarios

Mejora propuesta

- Implementar el registro unificado dentro del sistema.

Beneficio

- Módulos nuevos podrán integrarse fácilmente.
- Se evitan errores como los vistos en proyectos de compañeros.

Ampliar el módulo de servicios de usuario

Mejora propuesta

- Eliminar métodos que no corresponden al modelo de usuarios y moverlos a la capa de servicios de usuario.

Beneficio

- Menos duplicación de código.
- Facilita el mantenimiento del sistema de usuarios.

Gestión de Súper Usuarios

Mejora propuesta

- Ajustar el menú del súper usuario para mayor claridad.
- Usar correos institucionales @ucr.ac.cr como username.

Beneficio

- Navegación más eficiente.
- Mayor consistencia y fácil identificación de cuentas.

Pruebas Automáticas y CI/CD

Mejora propuesta

- Agregar pruebas automáticas para funcionalidades críticas.
- Configurar pipelines de CI/CD para validar y desplegar cambios.

Beneficio

- Menos errores al integrar nuevas funciones.
- Desarrollo más rápido, seguro y consistente.

Documentación y Estándares (PEP8)

Mejora propuesta

- Documentación completa del proyecto: módulos, flujos, configuraciones.
- Definir y aplicar estándares de codificación.

Beneficio

- Facilita mantenimiento y escalabilidad.
- Mejora la colaboración entre desarrolladores.

¡Gracias por su atención!